

Task 1 — Trace the Code

```
x = 5
y = 2
x = x + y
y = x * 2
x = y - x
print(x, y)
```

output: 7 14

Task 2 — Find the Bug

What does this program actually calculate? What was it probably intended to calculate? Fix it

- 1) This program calculates last value of the list
- 2) The total sum of the values

Fixed code:

```
numbers = [10, 20, 30, 40, 50]
total = 0
```

```
for i in range(len(numbers)):
    total += numbers[i]
```

```
print(total)
```

Task 3 — Nested Loops

```
for i in range(10):
    for j in range(10):
        print(i, j)
```

a) print() executes:

$10 \times 10 = 100$ times

b) Both loops from 0 to 999:

$1000 \times 1000 = 1,000,000$ times

c) Outer = 10, inner = 1,000,000:

$10 \times 1,000,000 = 10,000,000$ times

Task 4 — Which Is Faster?

Algorithm A:

n operations $\rightarrow O(n)$

Algorithm B:

$n \times n$ operations $\rightarrow O(n^2)$

Algorithm B grows much faster.

For $n = 1\,000,000$:

$A \approx 1,000,000$

$B \approx 1,000,000,000,000$

Task 5 — Complexity Ranking

From fastest-growing to slowest-growing:

2^n

n^2

$n \log_2 n$

n

$\log_2 n$

1

The most problematic are:

2^n and n^2

Especially 2^n , because it grows extremely quickly.

Task 6 — Searching

List:

[3, 8, 12, 17, 24, 31, 45, 51, 63]

1. Unsorted / unknown order: use linear search.

2. Sorted: use binary search.

Check the middle, eliminate half of the list each time.

Binary search is faster because it reduces the search space by half every step.

Linear: $O(n)$

Binary: $O(\log n)$

Task 7 — Large Dataset

Use one variable for the current maximum.

```
max_value = -infinity
```

```
for number in numbers:
    if number > max_value:
        max_value = number
```

Each number is examined **once**.

Time: $O(n)$

Memory: $O(1)$

You do **not** need to store all 500 million numbers. You can process them in chunks.

Task 8 — Counting

Use a dictionary/hash map.

```
counts = {}
```

```
for x in data:
    counts[x] = counts.get(x, 0) + 1
```

Result:

A → 4

B → 3

C → 2

D → 1

Data structure:

Dictionary / Hash Map

Task 9 — Matrix Operations

Given:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Addition:

$$A + B = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$$

Multiplication:

$$A \times B = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

For two 2x2 matrices:

$$2 \times 2 \times 2 = 8$$

So, 8 multiplication operations.

Task 10 — CPU vs Memory

Three possible reasons:

1. Data is too large for CPU cache.
2. Memory access is slow.
3. The program is waiting for RAM/memory bandwidth.

The problem may be memory-bound, not CPU-bound.

Task 11 — What Happens When You Run a Program?

/program - Operating system receives the request - Checks permissions and finds the executable - Loads the program into memory - Creates a process - CPU starts executing instructions - Program performs its work - Output is sent to the terminal

The OS manages CPU, memory, files, and other resources.

Task 12 — Four Programs

Yes, they can run at the same time.

CPU time depends on:

number of CPU cores + OS scheduler + program workload

With **one CPU core**, programs share the core using time slicing

Task 13 — The Slow Program

I do not fully agree.

If every part of the program becomes twice as fast, then:

10 hours → 5 hours

But in reality, some parts may be limited by:

I/O
memory
network
synchronization

other bottlenecks

So a computer that is "twice as fast" does not necessarily make the whole program exactly twice as fast.

Task 14 — Make It Faster

Possible approaches:

1. Improve the algorithm
2. Reduce unnecessary calculations
3. Use better data structures
4. Reduce loops
5. Reduce memory usage
6. Use caching
7. Use parallel processing
8. Use multiple CPU cores
9. Optimize database queries
10. Reduce disk/network I/O

The best first step is usually to find the actual bottleneck.

Task 15 — Bottleneck Identification

Original:

$$A = 2s$$

$$B = 2s$$

$$C = 50s$$

$$D = 3s$$

$$E = 2s$$

$$\text{Total} = 59s$$

After making A, B, D, E 10× faster:

$$A = 0.2s$$

B = 0.2s

C = 50s

D = 0.3s

E = 0.2s

New total:

50.9s

Speedup:

$59 / 50.9 \approx 1.16\times$

So the program becomes only about 16% faster.

Optimization effort should focus on **C**, because it takes 50 seconds.

Task 16 — 10 Million Student Marks

Because marks are only from 0 to 100, a counting array is very useful.

Keep:

count[0..100]

For every mark:

count[mark] += 1

At the same time calculate:

minimum

maximum

sum

number >= 90

number < 40

For the median, use the counts to find the middle position without sorting all 10 million values.

Data structures:

count[101] → frequency array

variables → min, max, sum, counters

Each mark is examined approximately once:

$O(n)$

Memory:

$O(101) \approx \text{constant}$

So we do **not** need to store all 10 million marks.

Likely bottleneck:

Reading/processing 10 million records

To make it faster:

- process input in chunks
- use efficient I/O
- parallelize processing if useful
- combine results from different chunks